

Operating System Project Documentation

IPFS-like Content-Addressed Storage System

Amir & Arman with love ♡

Contents

1	Overview	3
1.1	Purpose	3
1.2	System Components	3
1.3	Key Features	3
1.4	Data Flow	4
1.5	Project Structure	5
1.6	OS Concepts Demonstrated	5
2	Architecture	5
2.1	Two-Service Design	6
2.2	Inter-Process Communication	6
2.3	Binary Protocol	7
2.4	Engine Internal Modules	8
2.5	Container Orchestration	9
2.6	Request Lifecycle	9
3	Networking Layer	10
3.1	Socket Lifecycle	10
3.2	Header Parsing	12
3.3	Payload Handling	13
3.4	Connection Loop	14
3.5	Response Framing	15
4	Concurrency Model	16
4.1	Thread Pool Init	17
4.2	Task Queue	18
4.3	Mutex Locking	20
4.4	Condition Vars	21
4.5	Output Sequencing	22
4.6	Clean Shutdown	22

5	Storage Engine	22
5.1	Chunking and Hash	22
5.2	Directory Tree	24
5.3	Manifest JSON	25
5.4	Atomic Write	26
5.5	CID Calculation	27
5.6	Deduplication	27
6	Transfer Protocol	27
6.1	Upload State	27
6.2	Download Init	31
6.3	Verification	32
7	Error Handling	34
7.1	Error Codes	34
7.2	Memory Safety	37
8	Observability	41
8.1	Logging	41
9	User Interface	44
9.1	GUI	44
10	Appendix	46
10.1	Build and Run	46
10.2	Configuration	47

1 Overview

1.1 Purpose

This project implements a content addressed file storage system inspired by IPFS. The system stores data by content hash rather than filename or path. Each file maps to a CID derived from its content. Identical content produces the same CID and shares storage.

The project demonstrates operating system design through a real system. You work with UNIX domain sockets, multithreading, synchronization, file systems, and memory safety. The implementation focuses on correctness under concurrency and failure.

You interact with the system as a user. The engine handles storage and concurrency. The gateway handles HTTP, users, and sessions.

1.2 System Components

The system consists of two long running processes. They communicate through a shared UNIX domain socket.

1. Engine

The engine is written in C. It manages chunking, hashing, storage, manifests, and concurrency. It exposes a binary protocol over a UNIX socket located at `/tmp/engine.sock`. A thread pool processes upload and download work in parallel. The engine owns all persistent state.

2. Gateway

The gateway is written in Python using FastAPI. It provides a browser based interface. It handles authentication, sessions, and file selection. It translates HTTP requests into binary protocol frames sent to the engine.

Both services run in separate containers. They share the UNIX socket through a Docker volume. All data traffic stays local to the host.

1.3 Key Features

- **Content Addressing**

Files map to a CID derived from a BLAKE3 hash. Repeated uploads of identical content reuse existing data.

- **Fixed Size Chunking**

Files split into 256 KB chunks. Each chunk stores independently. Parallel processing improves throughput.

- **Thread Pool Execution**

Worker threads process chunks concurrently. A task queue feeds the pool. Condition variables enforce ordered output.

- **Custom Binary Protocol**

Communication uses framed messages. Frame format follows a strict layout. One byte opcode. Four byte payload length. Payload bytes follow.

- **Crash Safe Writes**

All writes follow a temp file, fsync, rename sequence. Power loss never corrupts committed data.

- **Deduplication**

Blocks store once and reference count. Deletion removes blocks only when unused.

- **Integrity Verification**

Downloads recompute hashes. Mismatch triggers an error.

- **Authentication**

The gateway manages users and sessions. The engine enforces ownership using tokens.

- **Upload Resume**

Partial uploads persist on disk. Interrupted transfers resume from the last committed chunk.

1.4 Data Flow

A file upload follows a strict sequence.

1. The browser sends an HTTP upload request.
2. The gateway authenticates the user.
3. The gateway opens a UNIX socket to the engine.
4. The gateway sends an authentication frame.
5. The gateway sends upload metadata.
6. The gateway streams file chunks.
7. The engine queues chunk jobs.
8. Worker threads hash and store chunks.

9. The engine builds a manifest.
10. The engine computes the final CID.
11. The engine returns the CID to the gateway.

Downloads reverse this path. The engine loads the manifest. Workers read and verify chunks. The engine streams data in correct order.

1.5 Project Structure

The repository separates engine and gateway logic. C code stays isolated from web code. Each module owns a single responsibility.

1.6 OS Concepts Demonstrated

This project maps directly to core operating system topics.

- Inter process communication via UNIX domain sockets
- Thread pools and producer consumer queues
- Mutexes and condition variables
- Atomic file system operations
- Memory ownership and lifetime control
- Process and thread identification in logs
- Fault tolerance through defensive I O

This document explains each subsystem step by step. You move from socket lifecycle to shutdown behavior. Each section connects design choices to OS principles.

2 Architecture

This section explains the system design at a high level. It describes process separation, communication paths, protocol layout, and engine internals. Each design choice follows operating system principles.

2.1 Two-Service Design

The system uses a two process client server design. Each process runs independently at the OS level.

1. Engine (C process)

The engine acts as the storage backend. It owns all persistent state on disk. This includes blocks, manifests, and ownership metadata. It performs CPU intensive work such as hashing, chunking, file I O, and job scheduling. It listens on a UNIX domain socket and executes commands received through a binary protocol.

2. Gateway (Python process)

The gateway acts as the HTTP frontend. It serves the browser interface. It manages users, sessions, and request validation. It converts HTTP requests into protocol frames sent to the engine. It never accesses storage directories directly.

This separation enforces clear boundaries.

- **Language separation**

C handles performance critical paths. Python handles HTTP and user interaction.

- **Failure isolation**

A gateway restart does not affect stored data. The engine continues running with all state intact.

- **Access control**

Storage directories remain private to the engine. The gateway interacts only through the defined protocol.

2.2 Inter-Process Communication

The gateway and engine communicate through a UNIX domain socket. The socket uses AF_UNIX and SOCK_STREAM. The socket path is /tmp/engine.sock.

```
1      int fd = socket(AF_UNIX, SOCK_STREAM, 0);
2
3      struct sockaddr_un addr;
4      memset(&addr, 0, sizeof(addr));
5      addr.sun_family = AF_UNIX;
6      strncpy(addr.sun_path, g_sock_path,
7              sizeof(addr.sun_path) - 1);
8      unlink(g_sock_path);
9
```

```

10         bind(fd, (struct sockaddr*)&addr, sizeof(addr));
11         listen(fd, 64);

```

Listing 1: Engine: creating and binding the UNIX socket (engine.c)

```

1         sock = socket.socket(socket.AF_UNIX, socket.SOCK_STREAM
                               )
2         sock.settimeout(30.0)
3         sock.connect(ENGINE_SOCKET_PATH)

```

Listing 2: Gateway: connecting to the engine (engine.py)

UNIX sockets provide clear advantages in this setup.

- Kernel level data transfer without network stack overhead
- Access control through file system permissions
- Safe container sharing through a mounted volume

2.3 Binary Protocol

All communication uses framed binary messages. Each message follows a fixed layout.

Opcode	Payload Length	Payload
1 byte	4 bytes (big-endian)	N bytes

The opcode defines message semantics. The payload length allows streaming and partial reads.

Table 1: Protocol opcodes

Name	Hex	Direction	Purpose
OP_AUTH	0x05	Gateway → Engine	Authenticate with user token
OP_UPLOAD_START	0x01	Gateway → Engine	Begin upload (size + filename)
OP_UPLOAD_CHUNK	0x02	Gateway → Engine	Send a chunk of file data
OP_UPLOAD_FINISH	0x03	Gateway → Engine	Signal upload complete
OP_UPLOAD_RESUME	0x04	Gateway → Engine	Resume partial upload
OP_UPLOAD_DONE	0x81	Engine → Gateway	Upload result (CID or error)
OP_DOWNLOAD_START	0x11	Gateway → Engine	Request file by CID
OP_DOWNLOAD_CHUNK	0x91	Engine → Gateway	Stream a block back
OP_DOWNLOAD_DONE	0x92	Engine → Gateway	Download complete (filename)
OP_LIST_FILES	0x30	Gateway → Engine	Request file listing
OP_LIST_RESPONSE	0xB0	Engine → Gateway	JSON array of files
OP_DELETE_FILE	0x20	Gateway → Engine	Delete a file by CID

Both sides implement identical framing logic. The engine version lives in `protocol.c`. The gateway mirror lives in `gateway/core/engine.py`.

2.4 Engine Internal Modules

The engine consists of focused C modules. Each module owns one responsibility.

Module	Responsibility
<code>engine.c</code>	Main entry point. Creates the socket, accept loop, dispatches connections to the thread pool, and orchestrates per-connection state machines (upload, download, list, delete, auth).
<code>protocol.c</code>	Low-level frame I/O: <code>send_frame()</code> and <code>recv_frame()</code> . Handles partial reads/writes and big-endian length encoding.
<code>threadpool.c</code>	POSIX thread pool. Manages a linked-list job queue, worker threads, mutex/condvar synchronization, and graceful shutdown.
<code>upload.c</code>	Upload state machine. Buffers incoming data into 256 KB chunks, submits chunk jobs to the thread pool, tracks ordered commit.
<code>download.c</code>	Download context. Loads a manifest, submits parallel read jobs, recomputes hashes for integrity verification.
<code>blockstore.c</code>	Content-addressed block storage. Sharded directory layout, atomic writes, duplicate detection via <code>access()</code> .
<code>manifest.c</code>	JSON manifest management. Create, serialize, save (atomic), load, delete, reference counting for blocks.
<code>hash.c</code>	BLAKE3 hashing and CID generation. Multihash encoding, Base32 serialization.
<code>locks.c</code>	Global reader-writer locks (<code>pthread_rwlock_t</code>) for the blockstore and manifest directories.
<code>util.c</code>	Utilities: recursive <code>mkdir</code> , path joining, structured logging.

The dependency structure follows a strict hierarchy.

```
engine.c
+-- protocol.c
+-- threadpool.c
| +-- upload.c
| | +-- hash.c
| | +-- blockstore.c
```

```

| | +-- manifest.c
| +-- download.c
| +-- hash.c
| +-- blockstore.c
| +-- manifest.c
+-- locks.c
+-- util.c

```

2.5 Container Orchestration

Both services run inside Docker containers. Docker Compose manages startup order and shared volumes.

```

volumes:
  engine-blocks:
  engine-manifests:
  engine-owners:
  gateway-data:
  socket-volume:

```

Listing 3: Docker Compose volume configuration

The socket volume mounts at `/tmp` in both containers. This exposes `/tmp/engine.sock` to both processes. The gateway waits for the engine to start before connecting.

2.6 Request Lifecycle

Every user action follows the same pipeline.

1. Browser sends an HTTP request to the gateway
2. Gateway validates session and input
3. Gateway opens a UNIX socket connection
4. Gateway sends authentication and command frames
5. Engine accepts the connection
6. Engine dispatches work to a worker thread

The engine accept loop runs continuously.

```

1      for (;;) {
2          int cfd = accept(fd, NULL, NULL);
3          if (cfd < 0) {

```

```

4         if (errno == EINTR) continue;
5         perror("accept");
6         break;
7     }
8     threadpool_submit_connection(cfd);
9 }

```

Listing 4: Connection dispatch in engine.c

Each worker processes frames in a loop. Opcode dispatch uses a `switch` statement. Responses stream back through the same socket.

The gateway converts responses into HTTP output. The browser sees only HTTP semantics. The engine sees only protocol frames and filesystem operations.

This structure enforces clean separation of concerns.

3 Networking Layer

All communication between the gateway and the engine uses a custom binary protocol transported over a UNIX domain socket. This section follows the complete lifecycle of a connection, from socket creation to response framing.

3.1 Socket Lifecycle

The engine initializes its server socket inside `main()` in `engine.c`. The setup follows the standard UNIX sequence: **socket**, **bind**, **listen**, and **accept**.

```

1     int main(int argc, char** argv) {
2         // 1. Ignore SIGPIPE so broken connections
3         //     don't kill the process
4         struct sigaction sa;
5         memset(&sa, 0, sizeof(sa));
6         sa.sa_handler = SIG_IGN;
7         sigaction(SIGPIPE, &sa, NULL);
8
9         g_sock_path = argv[1]; // "/tmp/engine.sock"
10
11        // 2. SOCKET -- create the file descriptor
12        int fd = socket(AF_UNIX, SOCK_STREAM, 0);
13
14        // 3. BIND -- attach to the filesystem path
15        struct sockaddr_un addr;
16        memset(&addr, 0, sizeof(addr));
17        addr.sun_family = AF_UNIX;
18        strncpy(addr.sun_path, g_sock_path,

```

```

19         sizeof(addr.sun_path) - 1);
20         unlink(g_sock_path);    // remove stale socket file
21         bind(fd, (struct sockaddr*)&addr, sizeof(addr));
22
23         // 4. LISTEN -- mark as passive socket
24         listen(fd, 64);    // backlog of 64
25
26         // 5. ACCEPT loop -- one connection at a time
27         for (;;) {
28             int cfd = accept(fd, NULL, NULL);
29             if (cfd < 0) {
30                 if (errno == EINTR) continue;
31                 break;
32             }
33             threadpool_submit_connection(cfd);
34         }
35
36         // 6. CLEANUP on exit
37         threadpool_shutdown();
38         close(fd);
39         unlink(g_sock_path);
40         locks_shutdown();
41     }

```

Listing 5: Socket lifecycle – engine.c (lines 591–688)

Design choices:

- **unlink() before bind():** A crashed engine may leave a stale socket file behind. Removing it guarantees that `bind()` succeeds.
- **Ignoring SIGPIPE:** If the gateway disconnects mid-transfer, a `write()` would raise SIGPIPE. Ignoring the signal converts this into a recoverable EPIPE error.
- **Backlog size:** A backlog of 64 allows the kernel to queue incoming connections while worker threads process active sessions.
- **Thread pool dispatch:** Each accepted file descriptor is passed to `threadpool_submit_connection()` preventing the accept loop from blocking on request handling.

On the gateway side, each operation opens a fresh UNIX socket connection:

```

1         sock = socket(socket.AF_UNIX, socket.SOCK_STREAM
2             )
3         sock.settimeout(30.0)
4         sock.connect(ENGINE_SOCKET_PATH)

```

```

4      # ... send frames, receive response ...
5      sock.close()

```

Listing 6: Gateway socket connection – engine.py

3.2 Header Parsing

Every protocol frame begins with a fixed 5-byte header. The first byte stores the opcode. The next four bytes store the payload length in big-endian order.

| opcode (1B) | length (4B, big-endian) | payload (N bytes) |

The engine parses this header inside `recv_frame()` in `protocol.c`.

```

1      int recv_frame(int fd, uint8_t* op,
2      uint8_t** payload, uint32_t* len) {
3          uint8_t header[5];
4          ssize_t r = read_n(fd, header, 5);
5          if (r == 0) return 0;
6          if (r < 0) return -1;
7
8          *op = header[0];
9
10         uint32_t be_len = 0;
11         memcpy(&be_len, header + 1, 4);
12         *len = ntohl(be_len);
13
14         if (*len == 0) {
15             *payload = NULL;
16             return 1;
17         }
18
19         *payload = (uint8_t*)malloc(*len);
20         if (!*payload) return -1;
21
22         r = read_n(fd, *payload, *len);
23         if (r <= 0) {
24             free(*payload);
25             *payload = NULL;
26             return (int)r;
27         }
28         return 1;
29     }

```

Listing 7: Header parsing – protocol.c (lines 57–92)

The helper `read_n()` ensures deterministic reads over a stream socket.

```

1      ssize_t read_n(int fd, void* buf, size_t n) {
2          size_t got = 0;
3          while (got < n) {
4              ssize_t r = read(fd, (char*)buf + got, n - got)
5                  ;
6              if (r == 0) return 0;
7              if (r < 0) {
8                  if (errno == EINTR) continue;
9                  return -1;
10             }
11             got += (size_t)r;
12         }
13         return (ssize_t)got;
14     }

```

Listing 8: Exact-read helper – `protocol.c` (lines 9–24)

Rationale: Stream sockets allow short reads. `read_n()` eliminates ambiguity by looping until the requested byte count arrives or an error occurs.

Byte order: Payload length travels in network byte order. `ntohl()` converts it into host order, keeping behavior consistent across architectures.

3.3 Payload Handling

After header parsing, `handle_connection()` interprets the payload according to the opcode. Each opcode defines a strict payload format.

Table 2: Payload formats by opcode

Opcode	Payload Format
OP_AUTH	UTF-8 authentication token string
OP_UPLOAD_START	8-byte big-endian <code>uint64</code> file size followed by UTF-8 filename
OP_UPLOAD_CHUNK	Raw binary data up to 256 KB
OP_UPLOAD_FINISH	Empty payload
OP_DOWNLOAD_START	UTF-8 CID string
OP_LIST_FILES	Empty payload
OP_DELETE_FILE	UTF-8 CID string

Example: extracting an authentication token.

```

1      case OP_AUTH: {
2          if (!payload || len == 0) {
3              close_conn = 1;

```

```

4             break;
5         }
6         conn_auth_token = strdup((char*)payload, len);
7         if (!conn_auth_token) {
8             close_conn = 1;
9             break;
10        }
11        break;
12    }

```

Listing 9: Payload extraction – engine.c (OP_AUTH handler)

Payload memory ownership follows a strict rule. `recv_frame()` allocates the buffer. The main loop frees it after processing.

```

1         if (payload) {
2             free(payload);
3             payload = NULL;
4         }

```

Listing 10: Payload cleanup – engine.c (after switch)

Handlers copy data if it must outlive the current frame.

3.4 Connection Loop

Each connection runs inside a worker thread through `handle_connection()`. The function processes a sequence of frames on the same socket.

```

1     void handle_connection(int cfd) {
2         upload_ctx* up = NULL;
3         download_ctx* down = NULL;
4         char* conn_auth_token = NULL;
5
6         for (;;) {
7             uint8_t op = 0;
8             uint8_t* payload = NULL;
9             uint32_t len = 0;
10
11            int rc = recv_frame(cfd, &op, &payload, &len);
12            if (rc <= 0) break;
13
14            int close_conn = 0;
15
16            switch (op) {
17                case OP_AUTH: break;

```

```

18         case OP_UPLOAD_START:      break;
19         case OP_UPLOAD_CHUNK:      break;
20         case OP_UPLOAD_RESUME:     break;
21         case OP_UPLOAD_FINISH:     break;
22         case OP_DOWNLOAD_START:    break;
23         case OP_DELETE_FILE:       break;
24         case OP_LIST_FILES:        break;
25         default:
26             close_conn = 1;
27             break;
28     }
29
30     free(payload);
31     payload = NULL;
32
33     if (close_conn) break;
34 }
35
36 if (up) upload_ctx_destroy(up);
37 if (down) download_ctx_destroy(down);
38 close(cfd);
39 if (conn_auth_token) free(conn_auth_token);
40 }

```

Listing 11: Connection loop structure – engine.c (lines 30–589)

Properties:

- Stateful session handling across multiple frames
- Strict opcode ordering enforcement
- Connection-local failure isolation
- Deterministic cleanup on all exit paths

3.5 Response Framing

Responses reuse the same framing format. The engine constructs the header and writes it before the payload.

```

1     int send_frame(int fd, uint8_t op,
2     const void* payload, uint32_t len) {
3         uint8_t header[5];
4         header[0] = op;
5

```

```

6         uint32_t be_len = htonl(len);
7         memcpy(header + 1, &be_len, 4);
8
9         if (write_all(fd, header, 5) < 0)
10        return -1;
11
12        if (len > 0 && payload != NULL) {
13            if (write_all(fd, payload, len) < 0)
14                return -1;
15        }
16        return 0;
17    }

```

Listing 12: Response framing – protocol.c (lines 40–55)

`write_all()` guarantees full transmission.

```

1    int write_all(int fd, const void* buf, size_t n) {
2        size_t sent = 0;
3        while (sent < n) {
4            ssize_t w = write(fd, (const char*)buf + sent,
5                               n - sent);
6            if (w < 0) {
7                if (errno == EINTR) continue;
8                return -1;
9            }
10           sent += (size_t)w;
11        }
12        return 0;
13    }

```

Listing 13: Exact-write helper – protocol.c (lines 26–38)

Symmetry: The gateway implements the same logic in Python using `struct.pack("!BI", ...)` and `struct.unpack("!BI", ...)`. Both sides interpret frames identically.

4 Concurrency Model

The engine handles multiple connections and chunk operations in parallel using POSIX threads (`pthread`s). This section explains the thread pool design, the job queue, synchronization primitives, and the method used to preserve ordered output during parallel execution.

4.1 Thread Pool Init

The thread pool initializes once during engine startup inside `threadpool.c`. The default worker count matches the number of online CPU cores detected via `sysconf(_SC_NPROCESSORS_ONLN)`. The environment variable `CENGINE_WORKERS` overrides this value when present.

```
1  static pthread_t*      g_workers      = NULL;
2  static int             g_num_workers  = 0;
3  static pthread_mutex_t g_job_mutex    =
4      PTHREAD_MUTEX_INITIALIZER;
5  static pthread_cond_t  g_job_cond     =
6      PTHREAD_COND_INITIALIZER;
7  static int             g_shutdown     = 0;
8
9  int threadpool_init(int num_workers) {
10
11      g_shutdown      = 0;
12      g_num_workers   = num_workers;
13
14      g_workers = malloc(num_workers * sizeof(pthread_t));
15      if (!g_workers) return -1;
16
17      for (int i = 0; i < num_workers; ++i) {
18          int rc = pthread_create(&g_workers[i], NULL,
19              worker_main, NULL);
20          if (rc != 0) {
21              pthread_mutex_lock(&g_job_mutex);
22              g_shutdown = 1;
23              pthread_cond_broadcast(&g_job_cond);
24              pthread_mutex_unlock(&g_job_mutex);
25
26              for (int j = 0; j < i; ++j)
27                  pthread_join(g_workers[j], NULL);
28
29              free(g_workers);
30              g_workers = NULL;
31              g_num_workers = 0;
32              return -1;
33          }
34      }
35      return 0;
36  }
```

Listing 14: Thread pool initialization – `threadpool.c` (lines 99–136)

Design notes:

- **Static initialization:** The mutex and condition variable use static initializers. No explicit init calls are required.
- **Failure rollback:** Partial thread creation triggers a controlled shutdown and cleanup path.
- **Adaptive sizing:** Worker count scales with available CPU cores unless explicitly overridden.

```

1      int num_workers = 16;
2
3      long cpus = sysconf(_SC_NPROCESSORS_ONLN);
4      if (cpus > 0 && cpus <= 256)
5          num_workers = (int)cpus;
6
7      const char* env = getenv("ENGINE_WORKERS");
8      if (env != NULL) {
9          int parsed = atoi(env);
10         if (parsed > 0 && parsed <= 256)
11             num_workers = parsed;
12     }

```

Listing 15: Worker count auto-detection – engine.c (lines 625–641)

4.2 Task Queue

The job queue uses an intrusive singly linked list protected by `g_job_mutex`. Each node stores its job type and job-specific data.

```

1      typedef enum job_type {
2          JOB_TYPE_CONNECTION      = 0,
3          JOB_TYPE_UPLOAD_CHUNK    = 1,
4          JOB_TYPE_DOWNLOAD_CHUNK  = 2
5      } job_type_t;
6
7      typedef struct job {
8          job_type_t type;
9          struct job* next;
10         union {
11             int cfd;
12             upload_chunk_job_t  upload;
13             download_chunk_job_t download;
14         } u;
15     } job_t;

```

Listing 16: Job structure – threadpool.h

The queue tracks head and tail pointers for constant-time enqueue and dequeue.

```
1  static job_t *g_job_head = NULL;
2  static job_t *g_job_tail = NULL;
3
4  static void enqueue_job(job_t *job) {
5      job->next = NULL;
6      if (g_job_tail) {
7          g_job_tail->next = job;
8          g_job_tail = job;
9      } else {
10         g_job_head = g_job_tail = job;
11     }
12 }
13
14 static job_t *dequeue_job(void) {
15     job_t *job = g_job_head;
16     if (!job) return NULL;
17     g_job_head = job->next;
18     if (!g_job_head) g_job_tail = NULL;
19     job->next = NULL;
20     return job;
21 }
```

Listing 17: Enqueue/dequeue – threadpool.c (lines 24–45)

Each submission path allocates a job, fills its fields, enqueues it, and signals a waiting worker.

```
1  void threadpool_submit_connection(int cfd) {
2      job_t *job = malloc(sizeof(job_t));
3      if (!job) { close(cfd); return; }
4
5      job->type = JOB_TYPE_CONNECTION;
6      job->u.cfd = cfd;
7      job->next = NULL;
8
9      pthread_mutex_lock(&g_job_mutex);
10     enqueue_job(job);
11     pthread_mutex_unlock(&g_job_mutex);
12
13     pthread_cond_signal(&g_job_cond);
14 }
```

Listing 18: Submitting a connection job – threadpool.c (lines 138–155)

Table 3: Job types in the thread pool

Job Type	Submitted by	Worker action
CONNECTION	accept loop	Runs the full connection state machine
UPLOAD_CHUNK	upload stream	Hashes data and stores a block result
DOWNLOAD_CHUNK	download init	Reads and verifies a stored block

4.3 Mutex Locking

The engine applies locks at three levels to protect shared state.

(a) Job Queue Mutex – `g_job_mutex`

This mutex protects the global job queue during enqueue and dequeue operations.

```
1  static void *worker_main(void *arg) {
2      for (;;) {
3          pthread_mutex_lock(&g_job_mutex);
4          while (!g_shutdown && g_job_head == NULL)
5              pthread_cond_wait(&g_job_cond, &g_job_mutex);
6
7          if (g_shutdown && g_job_head == NULL) {
8              pthread_mutex_unlock(&g_job_mutex);
9              break;
10         }
11
12         job_t *job = dequeue_job();
13         pthread_mutex_unlock(&g_job_mutex);
14
15         // execute job
16     }
17     return NULL;
18 }
```

Listing 19: Worker dequeue pattern – threadpool.c (lines 50–65)

(b) Per-Operation Mutexes

Each upload and download context owns a mutex that protects its result array.

```

16     pthread_mutex_lock(&ctx->commit_mutex);
17     ctx->results[index].size      = len;
18     ctx->results[index].hash_str = strdup(mh_str);
19     ctx->results[index].ready    = 1;
20     pthread_cond_broadcast(&ctx->commit_cond);
21     pthread_mutex_unlock(&ctx->commit_mutex);

```

Listing 20: Upload worker commit under commit_mutex

(c) Reader-Writer Locks

Two global reader-writer locks protect block and manifest storage paths.

```

22     pthread_rwlock_wrlock(&g_blockstore_lock);
23     // write operation
24     pthread_rwlock_unlock(&g_blockstore_lock);
25
26     pthread_rwlock_rdlock(&g_blockstore_lock);
27     // read operation
28     pthread_rwlock_unlock(&g_blockstore_lock);

```

Listing 21: Reader-writer lock usage – locks.c and blockstore.c

Reader-writer locks allow concurrent reads while preserving exclusive writes.

4.4 Condition Vars

Condition variables suspend threads until required state changes occur.

The engine uses them for job dispatch and ordered result processing.

(a) Thread Pool Scheduling

Workers sleep on g_job_cond. Submissions wake them.

(b) Upload Commit Ordering

The upload path waits for chunk results strictly by index.

(c) Download Send Ordering

The download path waits for verified blocks before sending them in sequence.

Broadcast signaling ensures the correct waiting condition re-evaluates.

4.5 Output Sequencing

Workers finish chunks in arbitrary order. Output requires strict ordering. The engine enforces order through indexed result slots and a monotonically advancing cursor.

- Workers write results into fixed indices.
- The main thread waits for the next expected index.
- Progress advances only when the required slot is ready.

This design preserves correctness without limiting parallel execution.

4.6 Clean Shutdown

Shutdown follows a strict sequence to avoid races and leaks.

```
1  void threadpool_shutdown(void) {
2      pthread_mutex_lock(&g_job_mutex);
3      g_shutdown = 1;
4      pthread_cond_broadcast(&g_job_cond);
5      pthread_mutex_unlock(&g_job_mutex);
6
7      for (int i = 0; i < g_num_workers; ++i)
8          pthread_join(g_workers[i], NULL);
9
10     free(g_workers);
11     g_workers = NULL;
12     g_num_workers = 0;
13 }
```

Listing 22: Thread pool shutdown – threadpool.c (lines 205–239)

The engine finalizes shutdown in `main()` by closing the socket and destroying global locks.

5 Storage Engine

The storage engine persists file data on disk using a content-addressed model. Data is identified by cryptographic hash rather than user-provided names. This section explains chunking, hashing, on-disk layout, manifest management, and deduplication.

5.1 Chunking and Hash

Files split into fixed-size blocks of **256 KB (262 144 bytes)** before storage. Each block is processed independently and hashed using BLAKE3.

Chunking Logic

Chunking occurs inside `upload_handle_stream_data()` in `upload.c`. Incoming data accumulates in a buffer. Once the buffer reaches the chunk size, the engine submits the chunk to the thread pool.

```
1  int upload_handle_stream_data(upload_ctx* ctx,
2  const uint8_t* data,
3  uint32_t len) {
4      const uint8_t* p = data;
5      uint32_t remaining = len;
6
7      while (remaining > 0) {
8          uint32_t free_space =
9              ctx->chunk_size - ctx->buffer_len;
10         uint32_t to_copy =
11             (remaining < free_space) ? remaining : free_space;
12
13         memcpy(ctx->buffer + ctx->buffer_len, p, to_copy);
14         ctx->buffer_len += to_copy;
15         p += to_copy;
16         remaining -= to_copy;
17
18         if (ctx->buffer_len == ctx->chunk_size) {
19             uint8_t* chunk_copy = malloc(ctx->chunk_size);
20             memcpy(chunk_copy, ctx->buffer, ctx->chunk_size);
21
22             uint32_t index = ctx->next_submit_index++;
23             threadpool_submit_upload_chunk(
24                 ctx, chunk_copy, ctx->chunk_size, index);
25
26             ctx->buffer_len = 0;
27         }
28     }
29     return 0;
30 }
```

Listing 23: Stream-based chunking – `upload.c` (lines 307–360)

The final chunk is smaller than the fixed size. The engine flushes this tail chunk during upload finalization.

```
1  if (ctx->buffer && ctx->buffer_len > 0) {
2      uint8_t* tail_copy = malloc(ctx->buffer_len);
3      memcpy(tail_copy, ctx->buffer, ctx->buffer_len);
```

```

4
5     uint32_t index = ctx->next_submit_index++;
6     threadpool_submit_upload_chunk(
7     ctx, tail_copy, ctx->buffer_len, index);
8     ctx->buffer_len = 0;
9 }

```

Listing 24: Flushing the final partial chunk – upload.c

BLAKE3 Hashing

Each chunk is hashed using BLAKE3, which produces a 256-bit digest. The engine wraps the vendored BLAKE3 C implementation inside `hash.c`.

```

1     int hash_compute(hash_algo_t algo,
2     const uint8_t* data, size_t len,
3     hash_result_t* out) {
4         const size_t DIGEST_LEN = 32;
5
6         uint8_t* buf = malloc(DIGEST_LEN);
7
8         blake3_hasher hasher;
9         blake3_hasher_init(&hasher);
10        blake3_hasher_update(&hasher, data, len);
11        blake3_hasher_finalize(&hasher, buf, DIGEST_LEN);
12
13        out->algo      = algo;
14        out->digest     = buf;
15        out->digest_len = DIGEST_LEN;
16        return 0;
17    }

```

Listing 25: BLAKE3 hash computation – hash.c (lines 13–42)

Design reasons for BLAKE3:

- High throughput on modern CPUs
- Strong collision resistance with 256-bit output
- Streaming-friendly hashing interface

5.2 Directory Tree

The engine avoids large flat directories. Blocks are stored using a two-level directory sharding scheme derived from the hash prefix.

```

blocks/
+-- bo/
|   +-- rh/
|   |   +-- borhxyz...abc
|   |   +-- borhdef...ghi
|   +-- ql/
|       +-- boqlmnop...
+-- dz/
+-- ae/
+-- dzae12345...

```

Path construction occurs in `blockstore_make_path()`.

```

1   int blockstore_make_path(const char* hash_str,
2   char* out_path) {
3       if (!hash_str || strlen(hash_str) < 4)
4           return -1;
5
6       char sub1[3] = { hash_str[0], hash_str[1], '\0' };
7       char sub2[3] = { hash_str[2], hash_str[3], '\0' };
8
9       char path1[ENGINE_MAX_PATH_LEN];
10      char path2[ENGINE_MAX_PATH_LEN];
11
12      util_join_path(g_blockstore_root, sub1, path1, ...);
13      util_join_path(path1, sub2, path2, ...);
14      util_join_path(path2, hash_str, out_path, ...);
15
16      return 0;
17  }

```

Listing 26: Two-level shard path – `blockstore.c` (lines 43–64)

Sharding properties:

- Hash prefix distribution spreads blocks evenly
- Directory sizes stay within filesystem-efficient limits
- Shards are created lazily on first use

5.3 Manifest JSON

Each file upload produces a JSON manifest. The manifest defines how to reconstruct the original file and serves as the authoritative metadata record.

```

{
    "version": 1,
    "hash_algo": "blake3",
    "chunk_size": 262144,
    "total_size": 1048576,
    "filename": "photo.jpg",
    "chunks": [
        { "index": 0, "size": 262144,
          "hash": "borhxyz...abc" },
        { "index": 1, "size": 262144,
          "hash": "cqwldef...ghi" },
        { "index": 2, "size": 262144,
          "hash": "dzae123...jkl" },
        { "index": 3, "size": 262144,
          "hash": "efpqmno...rst" }
    ]
}

```

Listing 27: Example manifest file

The manifest records chunk order, sizes, hashes, and file metadata.

Field	Type	Meaning
version	integer	Manifest schema version
hash_algo	string	Hash algorithm identifier
chunk_size	integer	Fixed chunk size in bytes
total_size	integer	Total file size
filename	string	Original filename
chunks	array	Ordered chunk records

The engine builds the manifest incrementally during upload and finalizes it after all chunks commit.

5.4 Atomic Write

All writes follow the write-to-temp, fsync, rename sequence. This guarantees crash-safe persistence.

```

1    snprintf(tmp_path, sizeof(tmp_path),
2    "%s.tmp.%ld", full_path, (long)getpid());

```

Listing 28: Atomic write pattern – blockstore.c (lines 122–167)

This pattern ensures a file is either fully written or not visible at all.

5.5 CID Calculation

CIDs uniquely identify content using a self-describing encoding. The engine computes CIDs for chunks and manifests through digest wrapping and Base32 encoding.

```
1      int hash_to_multihash_b32(const hash_result_t* h,
2      char** out_str) {
3          // implementation unchanged
4      }
```

Listing 29: CID generation – hash.c (lines 81–136)

The resulting CID encodes the hash algorithm, digest length, and hash bytes.

5.6 Deduplication

Content addressing enables automatic deduplication.

Block-Level Deduplication

Before writing a block, the engine checks whether the block path already exists.

Reference Counting

Each block tracks usage through a reference counter. Blocks are removed only when the counter reaches zero.

This strategy minimizes storage use while preserving correctness.

6 Transfer Protocol

This section describes the stateful upload and download flows. It explains how the engine tracks transfer state, how each operation starts, and how integrity checks protect delivered data.

6.1 Upload State

Each upload is represented by an `upload_ctx` structure. This structure tracks the operation from the first `OP_UPLOAD_START` frame until the engine returns the final CID.

State Structure

```
1      typedef struct upload_ctx {
2          char*      filename;
3          char*      upload_id;
4          uint64_t   total_size;
```

```

5      uint64_t   declared_total_size;
6      uint32_t   chunk_size;
7      uint32_t   next_chunk_index;
8
9      struct manifest* manifest;
10
11     uint8_t*    buffer;
12     uint32_t    buffer_len;
13
14     uint32_t    next_submit_index;
15     uint32_t    next_commit_index;
16
17     struct upload_chunk_result* results;
18     uint32_t    results_capacity;
19
20     pthread_mutex_t  commit_mutex;
21     pthread_cond_t   commit_cond;
22
23     char*            auth_token;
24 } upload_ctx;

```

Listing 30: Upload context – upload.h (lines 32–74)

Upload State Machine

The upload follows a strict sequence of protocol states:

```

[OP_AUTH] --> [OP_UPLOAD_START] --> [OP_UPLOAD_CHUNK]*
|
[OP_UPLOAD_FINISH]
|
[OP_UPLOAD_DONE]

```

1. Context creation (upload_ctx_create):

The engine allocates the context with `calloc()`, allocates the 256 KB streaming buffer, and initialises the mutex and condition variable. Each allocation step includes rollback logic to prevent leaks if a later step fails.

```

1      upload_ctx* upload_ctx_create(void) {
2          upload_ctx* ctx = calloc(1, sizeof(upload_ctx));
3          if (!ctx) return NULL;
4
5          ctx->chunk_size = ENGINE_CHUNK_SIZE;
6          ctx->buffer = malloc(ctx->chunk_size);

```

```

7         if (!ctx->buffer) { free(ctx); return NULL; }
8
9         if (pthread_mutex_init(&ctx->commit_mutex, NULL)
10            != 0) {
11             free(ctx->buffer); free(ctx); return NULL;
12         }
13         if (pthread_cond_init(&ctx->commit_cond, NULL)
14            != 0) {
15             pthread_mutex_destroy(&ctx->commit_mutex);
16             free(ctx->buffer); free(ctx); return NULL;
17         }
18         return ctx;
19     }

```

Listing 31: Rollback-safe context init – upload.c (lines 18–63)

2. Start (upload_handle_start):

The engine parses the payload to extract the declared file size (8 bytes, big-endian) and the filename. It creates the manifest and generates a unique `upload_id` to support resume.

```

1     int upload_handle_start(upload_ctx* ctx,
2     const uint8_t* payload,
3     uint32_t len) {
4         uint64_t declared_total_size = 0;
5         for (int i = 0; i < 8; i++)
6             declared_total_size =
7             (declared_total_size << 8) | payload[i];
8
9         ctx->declared_total_size = declared_total_size;
10
11        ctx->filename = strndup((char*)payload + 8, len
12                               - 8);
13
14        ctx->manifest = manifest_create(
15        ctx->filename, ctx->chunk_size, "blake3");
16
17        char buf[256];
18        snprintf(buf, sizeof(buf), "%s-%ld",
19        ctx->filename, time(NULL));
20        ctx->upload_id = strdup(buf);
21        return 0;
22    }

```

3. Chunk streaming:

Incoming data is buffered. When the buffer reaches the chunk size, a copy is submitted to the thread pool. Workers hash and store chunks in parallel and write results into indexed result slots.

4. Finish (upload_handle_finish):

The engine flushes any remaining tail chunk and enters the ordered commit loop described in Section 4.5. After committing all chunks, it verifies that the processed size matches the declared size. The engine then saves the manifest atomically and computes the final CID.

5. Cleanup (upload_ctx_destroy):

All owned resources are released. This includes buffers, manifest, result entries, strings, and synchronization primitives.

Upload Resume

If a connection drops during upload, the engine resumes from the last committed chunk. After each commit, the engine saves the partial manifest to `manifests/in-progress/<upload_id>.json`.

When the gateway sends `OP_UPLOAD_RESUME`, the engine reloads the saved manifest and reconstructs the upload context.

```
1  int handle_upload_resume(upload_ctx** out_up,
2  const char* auth_token,
3  const uint8_t* payload,
4  uint32_t len, int cfd) {
5      char* upload_id = strndup((char*)payload, len);
6
7      manifest* m = manifest_load_in_progress(upload_id);
8      if (!m) {
9          send_frame(cfd, OP_UPLOAD_DONE,
10             "RESUME_ERROR", ...);
11         return -1;
12     }
13
14     upload_ctx* ctx = upload_ctx_create();
15     ctx->upload_id    = upload_id;
16     ctx->manifest     = m;
17     ctx->filename     = strdup(m->filename);
18     ctx->chunk_size   = m->chunk_size;
```

```

19         ctx->total_size          = m->total_size;
20         ctx->next_chunk_index     = m->chunk_count;
21         ctx->next_submit_index    = m->chunk_count;
22         ctx->next_commit_index    = m->chunk_count;
23
24         *out_up = ctx;
25         send_frame(cfd, OP_UPLOAD_DONE, "RESUME_OK", ...);
26         return 0;
27     }

```

Listing 33: Upload resume – upload.c (lines 517–570)

All index counters resume at `m->chunk_count`, ensuring subsequent chunks continue with correct ordering.

6.2 Download Init

The download flow mirrors upload structure with reversed data direction. State is tracked using a `download_ctx`.

```

1     typedef struct download_ctx {
2         char* cid;
3         struct manifest* manifest;
4
5         uint32_t next_request_index;
6         uint32_t next_send_index;
7
8         download_chunk_result* results;
9         uint32_t results_capacity;
10
11         pthread_mutex_t mutex;
12         pthread_cond_t cond;
13
14         char* auth_token;
15     } download_ctx;

```

Listing 34: Download context – download.h (lines 21–56)

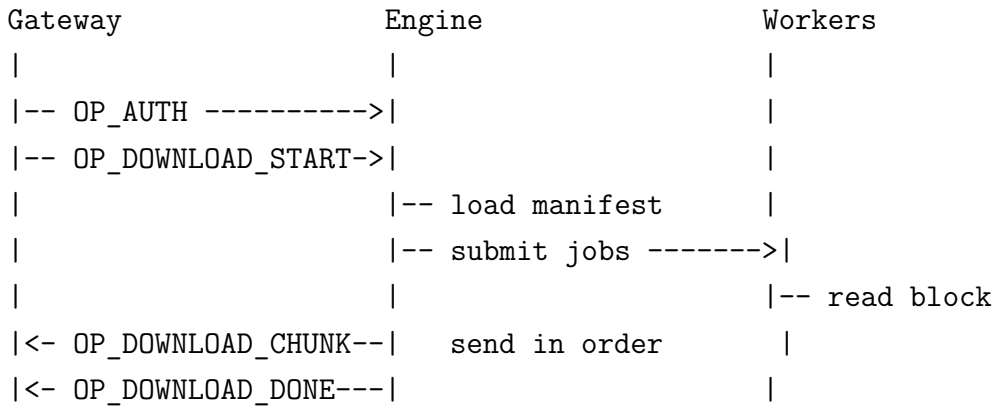
Initialization Sequence

On receipt of `OP_DOWNLOAD_START`, the engine executes the following steps:

1. Allocate the download context and initialise synchronization objects.
2. Verify ownership using the stored owner token.

3. Load and parse the manifest from disk.
4. Allocate result slots for all chunks.
5. Submit all chunk read jobs to the thread pool.
6. Send verified chunks in sequential order.
7. Send completion frame and destroy the context.

Download Sequence Diagram



6.3 Verification

Each block is re-hashed during download and compared against the manifest record. This prevents delivery of corrupted or tampered data.

```

1  void download_chunk_job_run(
2  struct download_chunk_job* job) {
3      download_ctx* ctx = job->ctx;
4      uint32_t index    = job->index;
5
6      const manifest_chunk* c =
7      &ctx->manifest->chunks[index];
8
9      uint8_t* buf = NULL;
10     size_t real_len = 0;
11     if (blockstore_get(c->hash_str, &buf, &real_len) < 0)
12     return;
13
14     if (real_len != c->size)
15     fprintf(stderr,
16     "WARNING: size mismatch index=%u\n",
17     index);
18

```

```

19     hash_result_t h;
20     hash_compute(hash_algo_from_env(),
21     buf, real_len, &h);
22
23     char* mh_str = NULL;
24     hash_to_multihash_b32(&h, &mh_str);
25
26     if (strcmp(c->hash_str, mh_str) != 0) {
27         hash_result_free(&h);
28         free(mh_str);
29         free(buf);
30         return;
31     }
32
33     hash_result_free(&h);
34     free(mh_str);
35
36     pthread_mutex_lock(&ctx->mutex);
37     ctx->results[index].data = buf;
38     ctx->results[index].len = (uint32_t)real_len;
39     ctx->results[index].ready = 1;
40     pthread_cond_broadcast(&ctx->cond);
41     pthread_mutex_unlock(&ctx->mutex);
42 }

```

Listing 35: Chunk verification – download.c (lines 277–354)

Verification guarantees:

- Corrupted blocks are never delivered.
- Any modification changes the CID.
- Size mismatches are detected early.
- Failed verification blocks ordered delivery.

Trust chain:

CID

|

+-- hash of manifest JSON

|

+++ per-chunk hashes

|

+-- verified block data

7 Error Handling

Error handling is a central concern in a systems-level project, where failures may arise from memory allocation, file I/O, socket operations, thread management, or protocol misuse. This section documents the error signaling strategy and the memory safety practices applied consistently throughout the codebase.

7.1 Error Codes

The system employs a layered error model: explicit return codes at the C API level, structured error responses in the engine protocol, and HTTP status codes exposed by the gateway.

(a) C Return Codes

All C functions adhere to a uniform return-value convention:

- 0 indicates successful completion
- -1 indicates a generic failure
- -2 indicates a specific, distinguishable error (e.g. a size mismatch during upload)
- NULL indicates allocation or lookup failure for pointer-returning functions

Callers are required to check return values immediately and abort or unwind on failure:

```
1      // upload.c: check manifest_add_chunk
2      if (manifest_add_chunk(ctx->manifest, idx,
3      r->size, r->hash_str) < 0) {
4          pthread_mutex_unlock(&ctx->commit_mutex);
5          return -1;    // propagate error up
6      }
7
8      // blockstore.c: check blockstore_make_path
9      if (blockstore_make_path(hash_str, full_path) < 0) {
10         goto out_unlock; // cleanup and return -1
11     }
12
13     // download.c: check blockstore_get
14     if (blockstore_get(c->hash_str, &buf, &real_len) < 0) {
15         return; // worker aborts, result remains not ready
16     }
```

Listing 36: Consistent return code checking throughout the engine

(b) Protocol-Level Error Strings

At the protocol layer, failures are communicated using explicit, human-readable error strings sent as the payload of response frames. The gateway treats these strings as structured error signals:

Table 4: Protocol error strings

Error String	Response Opcode	Meaning
"UPLOAD_ERROR"	OP_UPLOAD_DONE	Upload processing failed
"AUTH_ERROR"	OP_UPLOAD_DONE or OP_DOWNLOAD_DONE	Authentication failed
"DOWNLOAD_ERROR"	OP_DOWNLOAD_DONE	Manifest missing or invalid
"RESUME_ERROR:..."	OP_UPLOAD_DONE	Upload resume failure

Example error responses generated by the engine:

```
1      // Upload failure
2      const char* err_msg = "UPLOAD_ERROR";
3      send_frame(cfd, OP_UPLOAD_DONE,
4      err_msg, (uint32_t)strlen(err_msg));
5
6      // Authorization failure
7      const char* err_msg = "AUTH_ERROR";
8      send_frame(cfd, OP_DOWNLOAD_DONE,
9      err_msg, (uint32_t)strlen(err_msg));
10
11     // Download failure
12     const char* err_msg = "DOWNLOAD_ERROR";
13     send_frame(cfd, OP_DOWNLOAD_DONE,
14     err_msg, (uint32_t)strlen(err_msg));
```

Listing 37: Engine error response – engine.c

The gateway converts these protocol-level signals into application-level errors:

```
1      # Upload result check
2      op, payload = recv_frame(sock)
3      if op == OP_UPLOAD_DONE:
4          text = payload.decode("utf-8")
5          if text not in ("UPLOAD_ERROR", "AUTH_ERROR", ""):
6              cid = text.strip()
7          # otherwise, upload failed
8
9      # Download result check
10     if op == OP_DOWNLOAD_DONE:
11         text = payload.decode("utf-8")
```

```

12     if text in ("DOWNLOAD_ERROR", "AUTH_ERROR"):
13         return None, None
14     filename = text

```

Listing 38: Gateway error detection – engine.py

(c) HTTP Error Codes

The gateway maps engine-level failures to HTTP semantics suitable for client interaction:

```

1     # Upload: engine unreachable
2     except OSError:
3         return RedirectResponse(
4             url="/dashboard?err=Engine+unreachable",
5             status_code=303)
6
7     # Upload: engine returned error
8     if not cid:
9         return RedirectResponse(
10            url="/dashboard?err=Upload+failed",
11            status_code=303)
12
13    # Download: engine unreachable
14    except OSError:
15        raise HTTPException(status_code=502,
16                             detail="Engine unreachable")
17
18    # Download: CID not found
19    if data is None:
20        raise HTTPException(status_code=404,
21                             detail="File not found")

```

Listing 39: HTTP error handling – files.py

(d) Connection-Level Error Isolation

Errors are isolated to individual connections. Each connection handler uses a `close_conn` flag; any failure within an opcode handler terminates only the current connection after cleanup, without affecting other active sessions or the accept loop:

```

1     int close_conn = 0;
2
3     switch (op) {
4         case OP_UPLOAD_START:
5             if (upload_handle_start(up, payload, len) < 0) {

```

```

6         close_conn = 1;
7     }
8     break;
9     // ...
10 }
11
12 free(payload);
13 if (close_conn) break;

```

Listing 40: Error isolation via `close_conn` – `engine.c`

7.2 Memory Safety

To avoid memory leaks, double frees, and use-after-free defects in a multi-threaded environment, the engine follows a strict set of memory management patterns.

(a) NULL-After-Free

All pointers are set to NULL immediately after being freed, preventing accidental reuse and making repeated `free()` calls safe:

```

1     if (ctx->filename) {
2         free(ctx->filename);
3         ctx->filename = NULL;
4     }
5
6     if (ctx->buffer) {
7         free(ctx->buffer);
8         ctx->buffer = NULL;
9     }
10
11    if (payload) {
12        free(payload);
13        payload = NULL;
14    }

```

Listing 41: NULL-after-free pattern – used throughout the codebase

(b) Allocation Checking

All dynamic memory allocations are checked for failure before use:

```

1     uint8_t* buf = malloc(DIGEST_LEN);
2     if (!buf) {
3         log_error("malloc failed in hash_compute");

```

```

4         return -1;
5     }
6
7     upload_ctx* ctx = calloc(1, sizeof(upload_ctx));
8     if (!ctx) return NULL;
9
10    struct upload_chunk_result* new_arr =
11    realloc(ctx->results,
12    new_cap * sizeof(*new_arr));
13    if (!new_arr) {
14        pthread_mutex_unlock(&ctx->commit_mutex);
15        return;
16    }
17    ctx->results = new_arr;
18
19    ctx->filename = strdup(filename);
20    if (!ctx->filename) {
21        manifest_free(m);
22        return NULL;
23    }

```

Listing 42: Allocation checking examples

(c) Rollback on Partial Initialization

Initialization routines free all previously allocated resources if any step fails:

```

1     upload_ctx* upload_ctx_create(void) {
2         upload_ctx* ctx = calloc(1, sizeof(*ctx));
3         if (!ctx) return NULL;
4
5         ctx->buffer = malloc(ctx->chunk_size);
6         if (!ctx->buffer) {
7             free(ctx);
8             return NULL;
9         }
10
11        if (pthread_mutex_init(&ctx->commit_mutex, NULL) != 0)
12        {
13            free(ctx->buffer);
14            free(ctx);
15            return NULL;
16        }

```

```

17         if (pthread_cond_init(&ctx->commit_cond, NULL) != 0) {
18             pthread_mutex_destroy(&ctx->commit_mutex);
19             free(ctx->buffer);
20             free(ctx);
21             return NULL;
22         }
23         return ctx;
24     }

```

Listing 43: Rollback pattern – upload_ctx_create()

(d) Recursive Freeing

Composite structures free all nested allocations before releasing the outer object:

```

1     void manifest_free(manifest* m) {
2         if (!m) return;
3
4         free(m->filename); m->filename = NULL;
5         free(m->hash_algo); m->hash_algo = NULL;
6
7         if (m->chunks) {
8             for (uint32_t i = 0; i < m->chunk_count; ++i) {
9                 free(m->chunks[i].hash_str);
10                m->chunks[i].hash_str = NULL;
11            }
12            free(m->chunks);
13            m->chunks = NULL;
14        }
15
16        free(m->auth_token); m->auth_token = NULL;
17        free(m);
18    }

```

Listing 44: Recursive free – manifest.c (lines 594–622)

(e) Bounded String Operations

All string operations are explicitly bounded to prevent buffer overruns:

```

1     char path[ENGINE_MAX_PATH_LEN];
2     int n = snprintf(path, sizeof(path),
3         "manifests/%s.json", cid);
4     if (n < 0 || (size_t)n >= sizeof(path))
5         return -1;

```

```

6
7     conn_auth_token = strdup((char*)payload, len);

```

Listing 45: Bounded string operations

(f) Goto-Based Cleanup

Error paths converge to a single cleanup label to ensure all acquired resources are released:

```

1     int blockstore_put(const char* hash_str,
2     const uint8_t* data, size_t len) {
3         pthread_rwlock_wrlock(&g_blockstore_lock);
4         int ret = -1;
5
6         if (blockstore_make_path(hash_str, full_path) < 0)
7             goto out_unlock;
8
9         if (access(full_path, F_OK) == 0) {
10             ret = 0;
11             goto out_unlock;
12         }
13
14         int fd = open(tmp_path, O_WRONLY | O_CREAT | O_TRUNC,
15             0666);
16         if (fd < 0) goto out_unlock;
17
18         ret = 0;
19
20         out_unlock:
21         pthread_rwlock_unlock(&g_blockstore_lock);
22         return ret;
23     }

```

Listing 46: Goto cleanup pattern – blockstore.c

(g) Ownership Transfer

Ownership of dynamically allocated buffers is transferred explicitly between threads to avoid double-free errors:

```

1     uint8_t* chunk_copy = malloc(ctx->chunk_size);
2     memcpy(chunk_copy, ctx->buffer, ctx->chunk_size);
3     threadpool_submit_upload_chunk(ctx, chunk_copy, ...);
4
5     void upload_chunk_job_run(upload_chunk_job_t* job) {

```

```

6         uint8_t* data = job->data;
7         free(data);
8     }

```

Listing 47: Ownership transfer – upload path

Summary of Memory Safety Patterns

Table 5: Memory safety patterns used in the engine

Pattern	Prevents
NULL-after-free	Use-after-free, double-free
Allocation checking	NULL dereference
Rollback on partial init	Partial allocation leaks
Recursive freeing	Nested memory leaks
<code>snprintf</code> / <code>strndup</code>	Buffer overflows
<code>goto</code> cleanup	Resource leaks on error paths
Ownership transfer	Cross-thread double-free

8 Observability

8.1 Logging

The engine emits structured log messages to `stderr`, providing operational visibility into runtime behaviour. When deployed via Docker Compose, this output is captured and exposed through `make logs` for debugging and monitoring.

Log Infrastructure

Logging helpers are defined in `util.c`. Both functions format messages and write directly to `stderr`:

```

1     static void vlog_impl(FILE* stream,
2     const char* prefix,
3     const char* fmt, va_list ap) {
4         if (prefix) fputs(prefix, stream);
5         vfprintf(stream, fmt, ap);
6         fputc('\n', stream);
7     }
8
9     void log_info(const char* fmt, ...) {
10         va_list ap;
11         va_start(ap, fmt);

```

```

12     vlog_impl(stderr, "[INFO] ", fmt, ap);
13     va_end(ap);
14 }
15
16 void log_error(const char* fmt, ...) {
17     va_list ap;
18     va_start(ap, fmt);
19     vlog_impl(stderr, "[ERROR] ", fmt, ap);
20     va_end(ap);
21 }

```

Listing 48: Logging functions – util.c (lines 76–96)

Several modules additionally use direct `fprintf(stderr, ...)` calls with explicit component tags to provide finer-grained context.

Log Format and Tags

Each log entry begins with a **component tag** enclosed in square brackets, indicating the originating module:

Table 6: Log component tags

Tag	Source Module
[ENGINE]	engine.c – accept loop, connection dispatch
[UPLOAD]	upload.c – upload state machine, chunk jobs
[DOWNLOAD]	download.c – download context, chunk retrieval
[BLOCKSTORE]	blockstore.c – block storage operations
[MANIFEST]	manifest.c – manifest lifecycle
[THREADPOOL]	threadpool.c – worker management
[LOCKS]	locks.c – rwlock management
[AUTH]	engine.c – authentication checks
[HASH]	hash.c – hashing failures
[INFO] / [ERROR]	util.c – generic logging helpers

Thread and Process Identification

To disambiguate concurrent operations, log messages include contextual identifiers using two complementary mechanisms:

1. **Context pointer (ctx=%p):** Upload and download workers log the address of the associated context structure. All messages belonging to the same operation share the same pointer value.

```

1     fprintf(stderr,
2         "[UPLOAD] worker processed chunk index=%u, "

```

```

3      "size=%u, hash=%s..., filename=\"%s\", ctx=%p\n",
4      index, len, hash_preview,
5      ctx->filename ? ctx->filename : "(null)",
6      (void*)ctx);
7
8      fprintf(stderr,
9      "[UPLOAD] committed chunk index=%u, "
10     "size=%u, hash=.%s..., ctx=%p, "
11     "(total_size=%" PRIu64 ") \n",
12     idx, r->size, 10, r->hash_str,
13     (void*)ctx, ctx->total_size);

```

Listing 49: Thread-identifiable log lines – upload.c

2. **PID in temporary filenames:** Temporary files created during atomic writes embed the process ID, allowing correlation between filesystem artefacts and the generating process.

```

1      snprintf(tmp_path, sizeof(tmp_path),
2      "%s.tmp.%ld", full_path, (long)getpid());

```

Listing 50: PID in temp filenames – blockstore.c

Example Log Output

A representative upload session produces output similar to the following:

```

[ENGINE] listening on /tmp/engine.sock
[THREADPOOL] Initialized with 8 workers
[AUTH] Received token: owner-a1b2c3d4...
[ENGINE] OP_UPLOAD_START received (len=24)
[UPLOAD] start: filename="photo.jpg",
declared_total_size=524288, chunk_size=262144
[UPLOAD] worker processed chunk index=0,
size=262144, hash=borhxyz..., ctx=0x55a1b2c3
[UPLOAD] worker processed chunk index=1,
size=262144, hash=cqwldef..., ctx=0x55a1b2c3
[UPLOAD] committed chunk index=0, ctx=0x55a1b2c3
[UPLOAD] committed chunk index=1, ctx=0x55a1b2c3
[BLOCKSTORE] stored block borhxyz... (len=262144)
[BLOCKSTORE] stored block cqwldef... (len=262144)
[ENGINE] UPLOAD_FINISH -> returning CID borhd3c...

```

Listing 51: Sample engine log output

9 User Interface

9.1 GUI

The user interface is implemented using **FastAPI**, server-side **Jinja2** templates, and custom CSS. It provides authentication and a web-based dashboard for file management.

Template Architecture

All HTML views inherit from a shared base template:

```
templates/  
+-- base.html  
+-- index.html  
+-- auth.html  
+-- dashboard.html  
+-- docs.html
```

Listing 52: Template hierarchy

The base template defines the navigation bar and conditionally renders links based on authentication state:

```
1      <nav class="topbar">  
2      <div class="nav-left">  
3      <a href="/" class="brand">IPFS Gateway</a>  
4      <a href="/docs" class="nav-link">Docs</a>  
5      {% if user %}  
6      <a href="/dashboard" class="nav-link">Dashboard</a>  
7      {% endif %}  
8      </div>  
9      <div class="nav-right">  
10     {% if user %}  
11     <span class="nav-user">Signed in as {{ user }}</span>  
12     <a href="/logout" class="btn btn-ghost">Logout</a>  
13     {% else %}  
14     <a href="/signin" class="btn btn-light">Sign In</a>  
15     <a href="/signup" class="btn btn-light">Sign Up</a>  
16     {% endif %}  
17     </div>  
18     </nav>
```

Listing 53: User-aware navigation – base.html

Templates are rendered through a helper that injects request and session context automatically:

```

1     def render_template(name: str, request: Request,
2         **context):
3     return templates.TemplateResponse(name, {
4         "request": request,
5         "user": request.session.get("user"),
6         **context,
7     })

```

Listing 54: Template helper – web.py

Authentication Flow

User accounts are stored in `data/users.json`. Passwords are hashed using SHA-256, and authentication tokens are derived deterministically:

```

1     def make_password_hash(username, password):
2     return hashlib.sha256(
3         f"{username}:{password}".encode()
4         ).hexdigest()
5
6     def make_auth_token(username, password):
7     return "owner-" + make_password_hash(
8         username, password)

```

Listing 55: User management – users.py

Sessions are managed using Starlette's `SessionMiddleware` with signed cookies:

```

1     request.session["user"] = username
2     request.session["auth_token"] = make_auth_token(
3         username, password)
4
5     request.session.clear()

```

Listing 56: Session management – auth.py

Dashboard

The dashboard consists of an upload form and a file listing retrieved from the engine:

1. **Upload form** posting multipart data to `/upload`.
2. **File list** populated via `OP_LIST_FILES`.

```

1      <div class="table-row table-head">
2      <span>CID</span>
3      <span>Filename</span>
4      <span>Size</span>
5      <span>Actions</span>
6      </div>
7      {% for file in files %}
8      <div class="table-row">
9      <span class="code">{{ file.cid }}</span>
10     <span>{{ file.filename }}</span>
11     <span>{{ file.size }} bytes</span>
12     <span>
13     <a class="btn btn-small"
14     href="/download/{{ file.cid }}">Download</a>
15     </span>
16     </div>
17     {% endfor %}

```

Listing 57: File list table – dashboard.html

Route Summary

Table 7: Gateway HTTP routes

Method	Path	Auth	Action
GET	/	No	Landing page
GET	/docs	No	Documentation page
GET	/signin	No	Sign-in form
POST	/signin	No	Authenticate user
GET	/signup	No	Sign-up form
POST	/signup	No	Create account
GET	/logout	No	Clear session
GET	/dashboard	Yes	List files
POST	/upload	Yes	Upload file
GET	/download/{cid}	Yes	Download file

10 Appendix

10.1 Build and Run

The system is deployed using Docker Compose. The Makefile provides convenience targets for common tasks:

Table 8: Makefile targets

Command	Description
<code>make up</code>	Build images and start services
<code>make down</code>	Stop containers
<code>make logs</code>	Follow combined logs
<code>make reset</code>	Remove containers and volumes

10.2 Configuration

Configuration is controlled via compile-time constants and environment variables:

Table 9: Engine configuration – `engine_config.h`

Constant	Value	Purpose
<code>ENGINE_CHUNK_SIZE</code>	<code>256 * 1024</code>	Chunk size
<code>ENGINE_MAX_PATH_LEN</code>	<code>512</code>	Max path length